

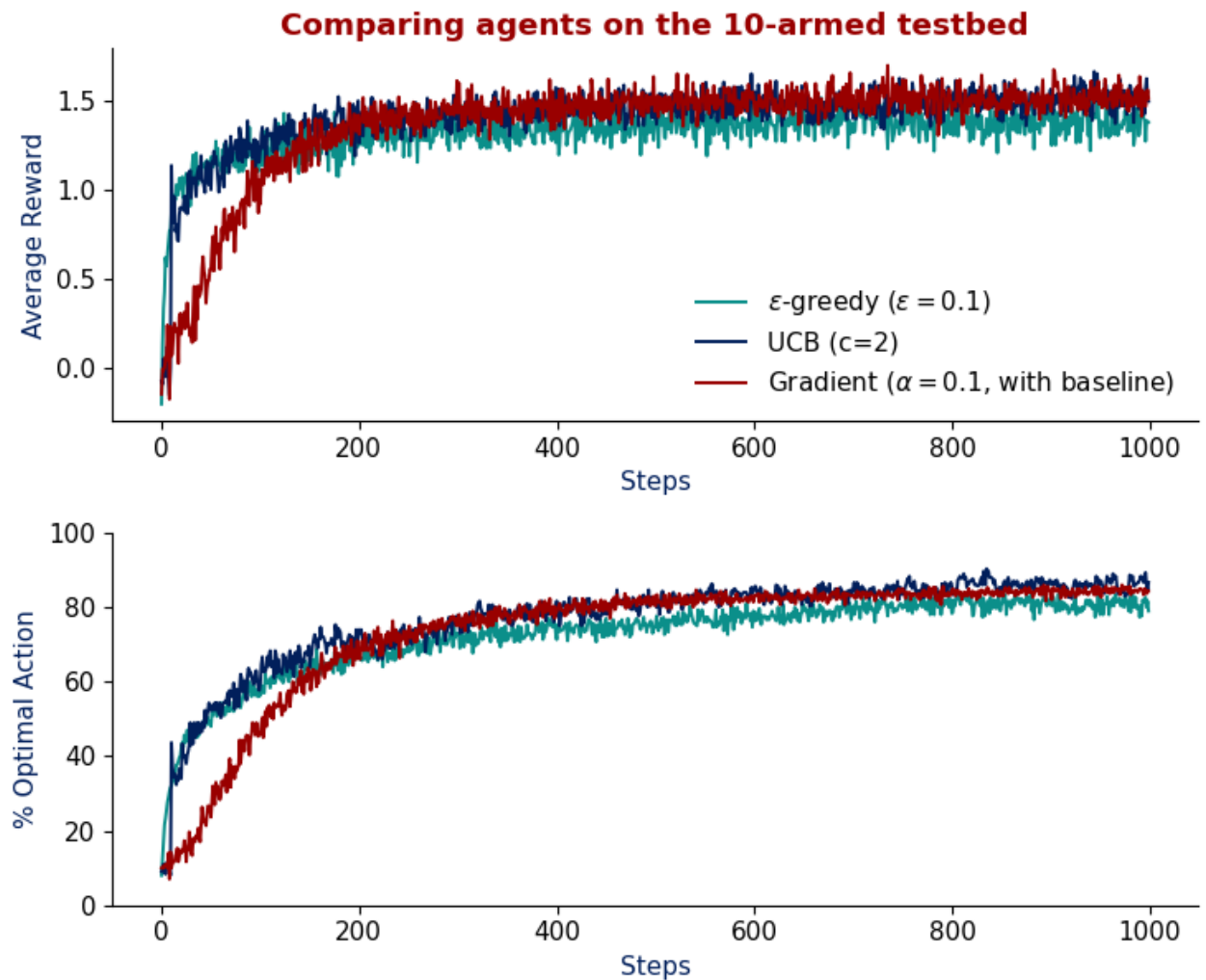
HW Problem 3

a): ϵ -greedy, UCB, and gradient results compared

```
In [34]: def compare_three(eps, c, alpha, out_file):
    labels = [f"$\\varepsilon$-greedy ($\\varepsilon$={eps})", f"UCB (c={c})",
    colors = [TEAL, PENN_BLUE, PENN_RED]
    agents = [epsilon_greedy_agent(eps), ucb_agent(c), gradient_bandit_agent(alpha)]
    results = [run_agent(a, steps=1000, runs=300, seed=11) for a in agents]

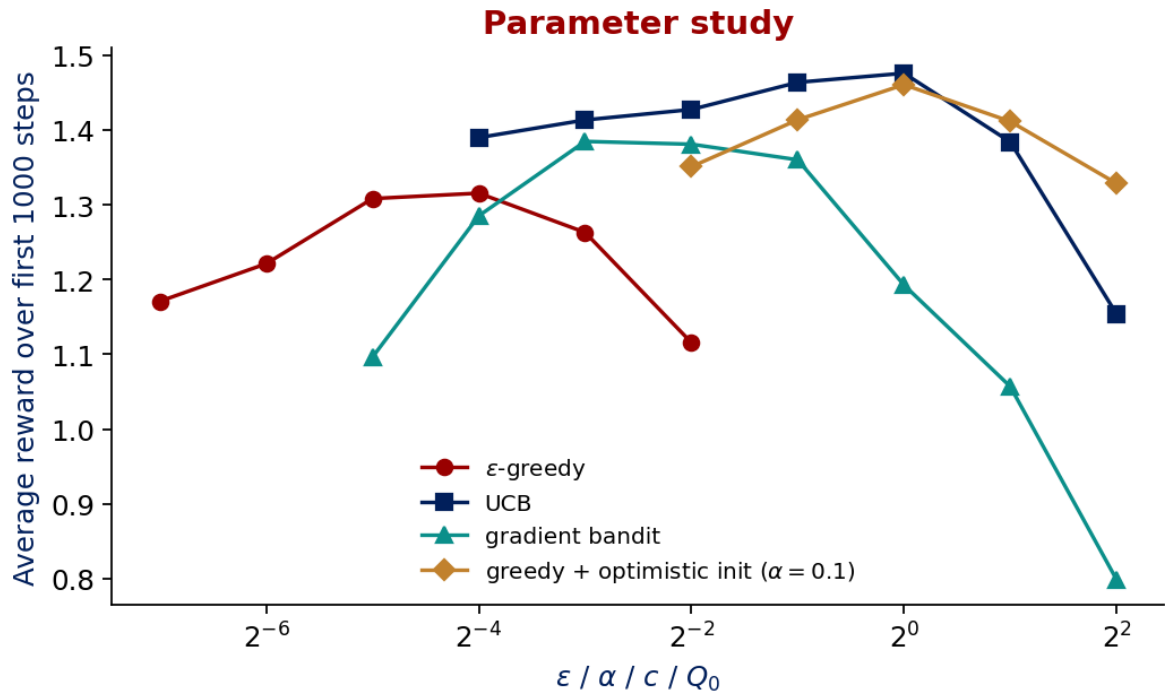
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7.5, 6.2))
    for label, result, color in zip(labels, results, colors):
        ax1.plot(result[0], color=color, label=label, linewidth=1.4)
        ax2.plot(100 * result[1], color=color, linewidth = 1.4)
    ax1.set_ylabel("Average Reward")
    ax1.legend(frameon=False, loc="lower right")
    ax1.set_xlabel("Steps")
    ax1.set_title("Comparing agents on the 10-armed testbed")
    ax2.set_ylabel("% Optimal Action")
    ax2.set_xlabel("Steps")
    ax2.set_ylim(0, 100)
    plt.tight_layout()
    plt.savefig(FIG_DIR / out_file)
    plt.show()

compare_three(0.1, 2, 0.1, "ps3_a.png")
```



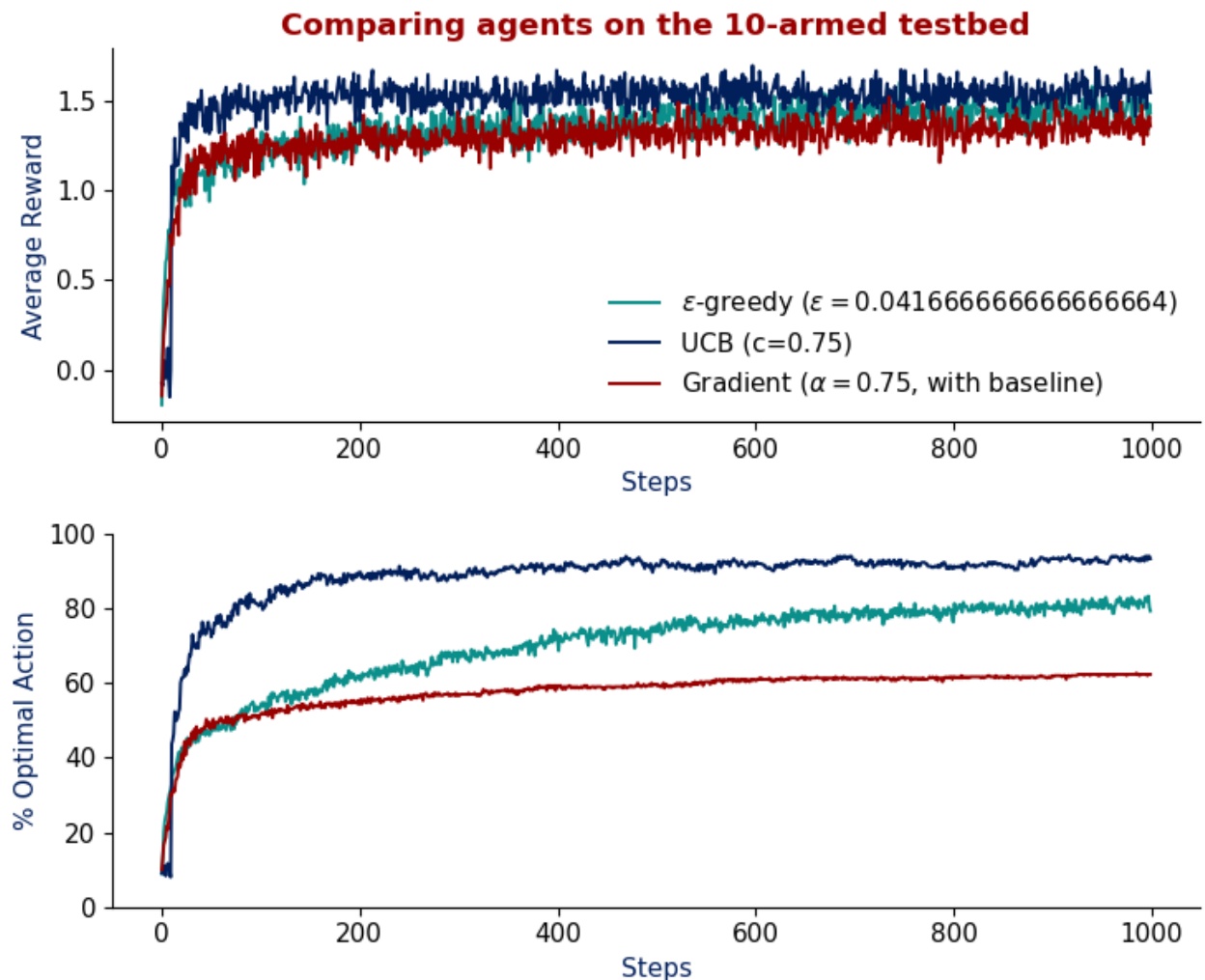
The gradient and UCB approaches both appear to reach a similarly high average reward, though the gradient bandit takes longer to learn than the other methods. UCB also appears to have the highest optimal action percentage, indicating that for this testbed & chosen metaparameters, it seems to be the ideal choice.

b): Tweaking Metaparameters



Looking at the original parameter study curves, it seems each has a local maxima. In an attempt to improve the average reward, I want to try and pick untested values near those maxima to see if there are better options hiding "between" the sampled points. Since the parameter study is looking at average reward across all 1000 steps and the existing agents are most likely converging to near-optimal policies, I would expect to see an improvement in convergence rate for all three approaches (if the parameters I'm selecting are actually improvements- otherwise I would expect a slower convergence rate.)

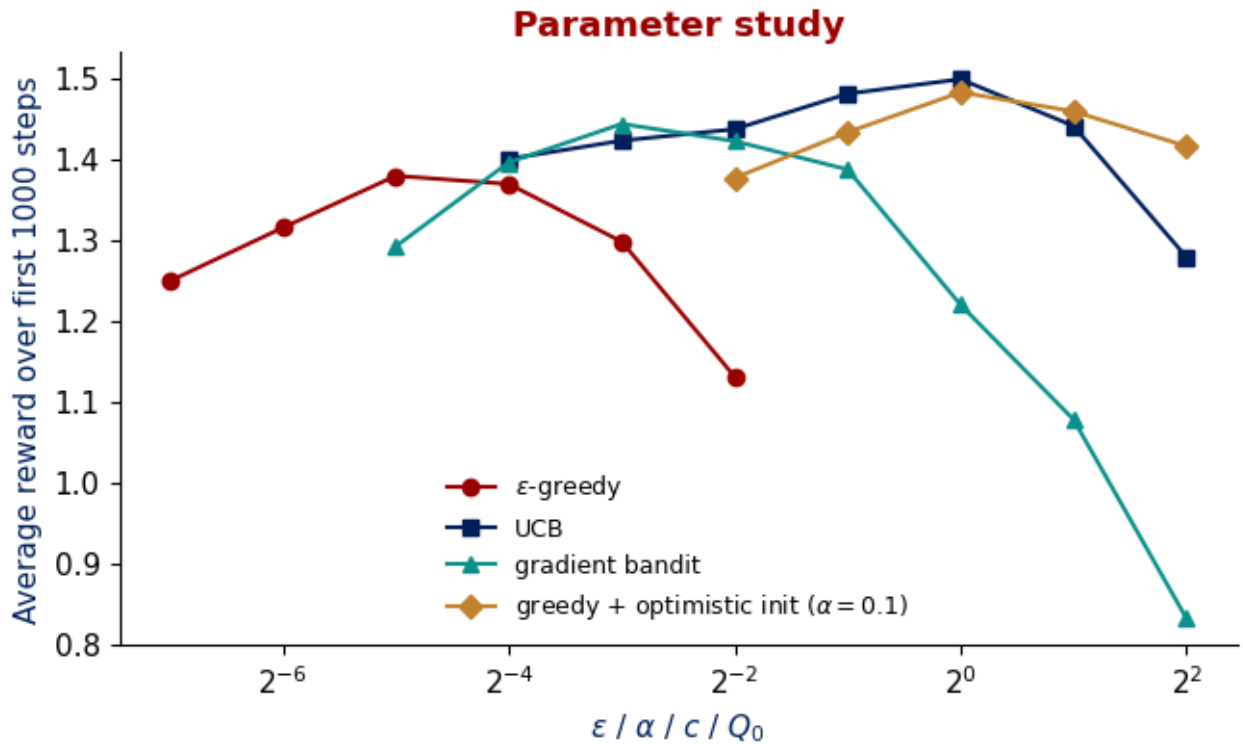
```
In [35]: compare_three(1/24, 0.75, 0.75, "ps3_b.png")
```



My hypothesis that the agents would converge faster appears to have been correct - this is evident by the much steeper curve in the first few steps that levels off quickly.

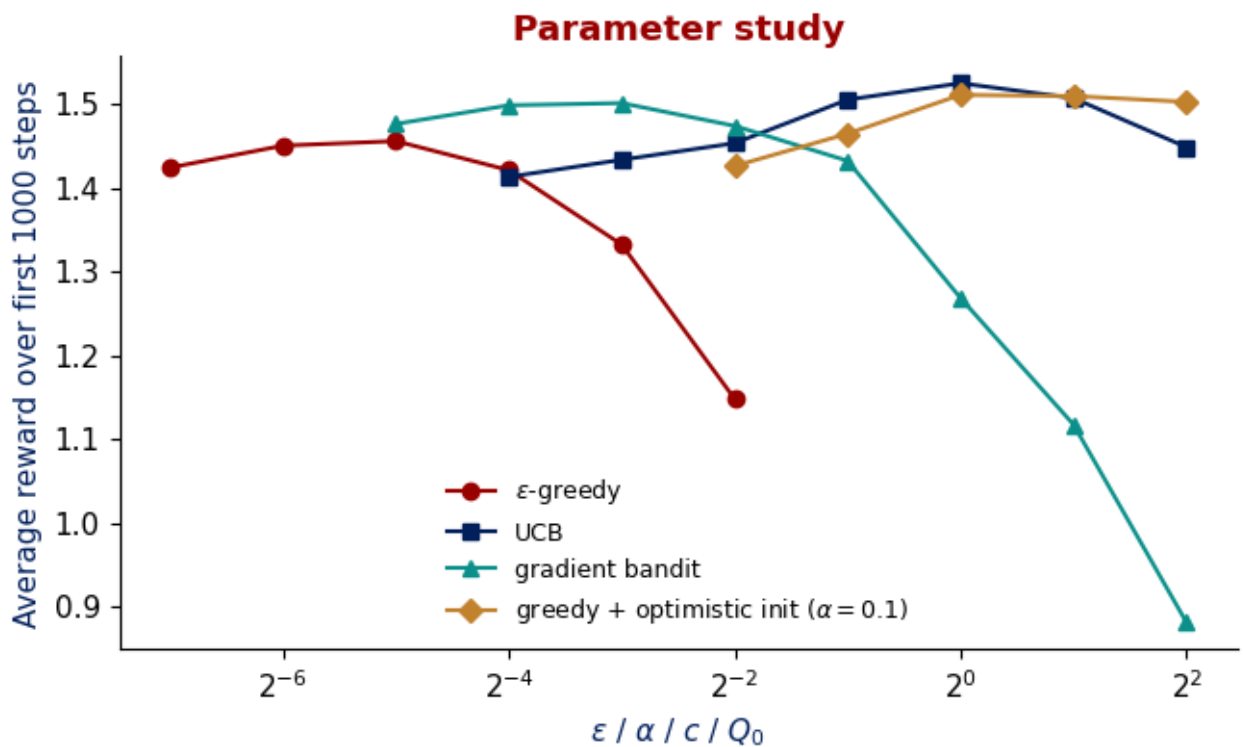
What I did not anticipate (but in hindsight should have) the % optimal action decreased dramatically for both the ϵ -greedy and gradient agents. In fact, the gradient agent now converges to a lower average return than the ϵ -greedy agent. This is representative of the inherent tradeoff between quick convergence and finding an optimal policy - when these agents converge, they are ending exploration that might lead to a better result. This does lead to a higher average return over the first 1000 steps, at the expense of the actual reward the agent learns to receive. This makes me suspect that if we ran our parameter study for a larger number of steps, we would "penalize" the agents less for the time they take to learn since we give them more time to reap the advantage of having found a higher return arm.

```
In [38]: parameter_study(runs=200, steps=2000, file_suffix="_2k")
```



This does seem to shift the curves, but not as significantly as expected. I think that I would need to run the study even longer for the average reward to not be dragged down by the early steps where the agents are exploring.

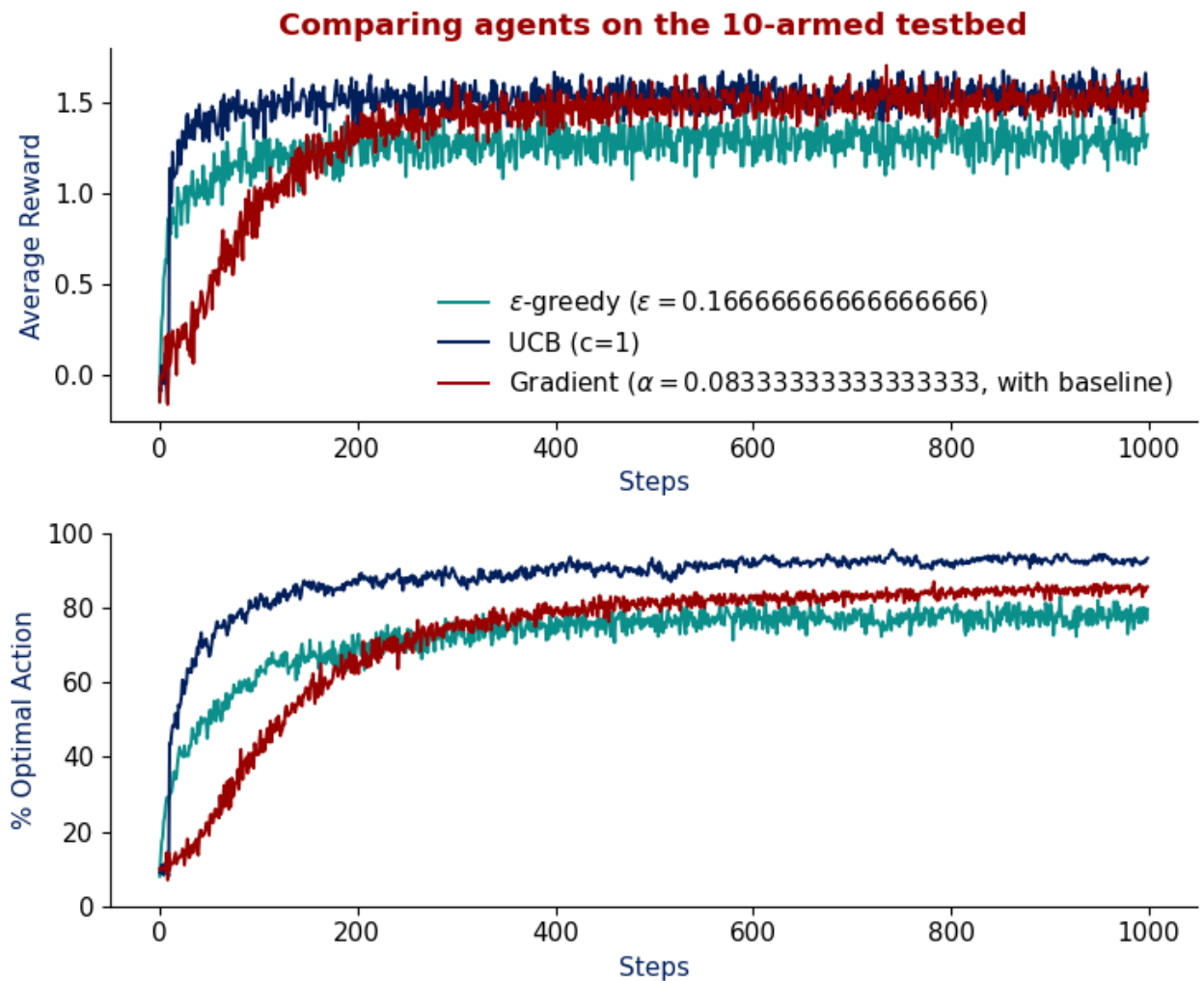
In [39]: `parameter_study(runs=200, steps=10000, file_suffix="_10k")`



As I suspected, the local maxima of the curves has shifted. I would expect values near

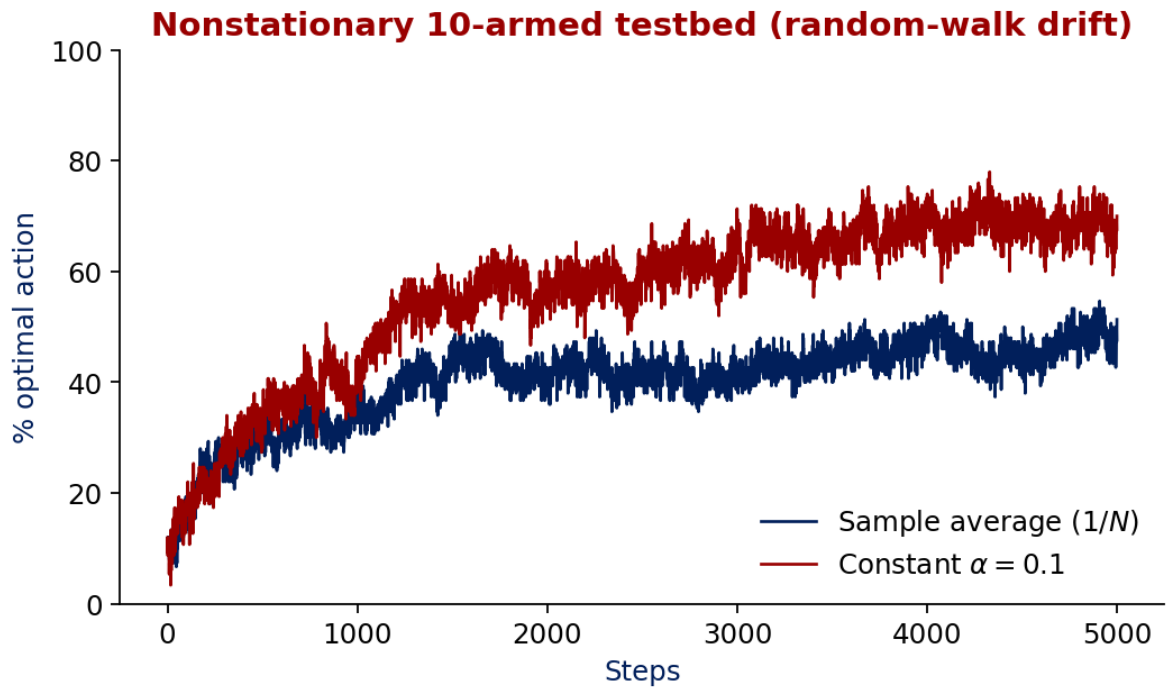
these maxima to converge slower than my last test, but have higher % optimal action.

```
In [43]: compare_three(1/6, 1, 1/12, "ps3_b2")
```



As expected, when compared to the previous test, the convergence rate for the gradient agent has decreased but the % optimal action has increased.

c): Nonstationary



As discussed in problem 1a, the sample average approach's step size decreases over time, making it less capable of adjusting to the change in observed rewards. This explains the fact that as time continues, the constant α approach outperforms the sample average.

If I were choosing an agent to deploy in an environment I knew ahead of time to be perpetually non-stationary, I would likely use a gradient agent, using the average observed reward as a baseline. My rationale is that with a well chosen α , we can expect the agent to continue to adjust its preferences as the rewards drift. My concern with UCB is that while its bonus factor does encourage continued exploration over time, the asymptotic nature of $\log(t)$ means that as t approaches infinity, $N_t(s)$ will dominate the bonus factor and reduce it too far to sufficiently introduce exploration (similar to the issues with sample averaging), whereas the constant α used in the gradient approach will remain constant.